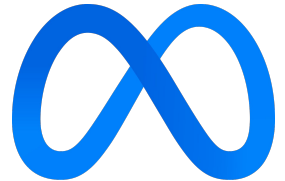
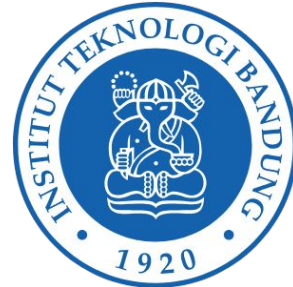
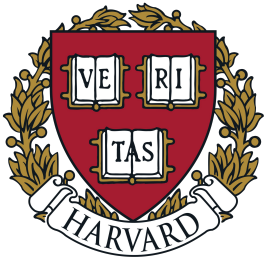


Learning-Augmented Heuristics: Simple Yet Smart, Robust and Interpretable Cache Eviction

Haocheng Xia¹², William Nixon³¹, Bintang Dwi Marthen¹⁴,
Pranav Bhandari⁵, Juncheng Yang¹



Caching and Eviction Algorithms

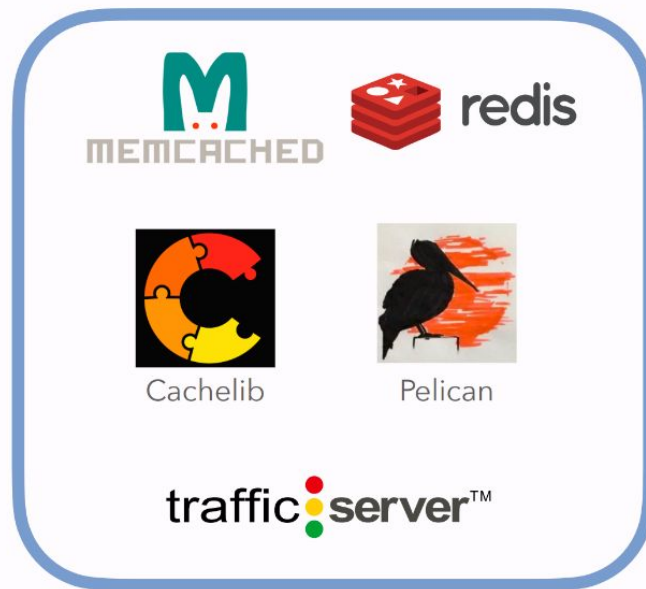
Software caches are ubiquitous.

Efficient eviction algorithms are key to better utilization of space.

Cache Metrics:

- Efficiency: Miss Ratio
- Throughput
- Simplicity

Page Cache



Web Caches

Evolution of Cache Eviction Algorithms

Various spectrums of eviction policies have been explored:

Static heuristics

FIFO · LRU · 2Q · S3-FIFO · SIEVE

Manually curated rules with
workload specific assumptions.

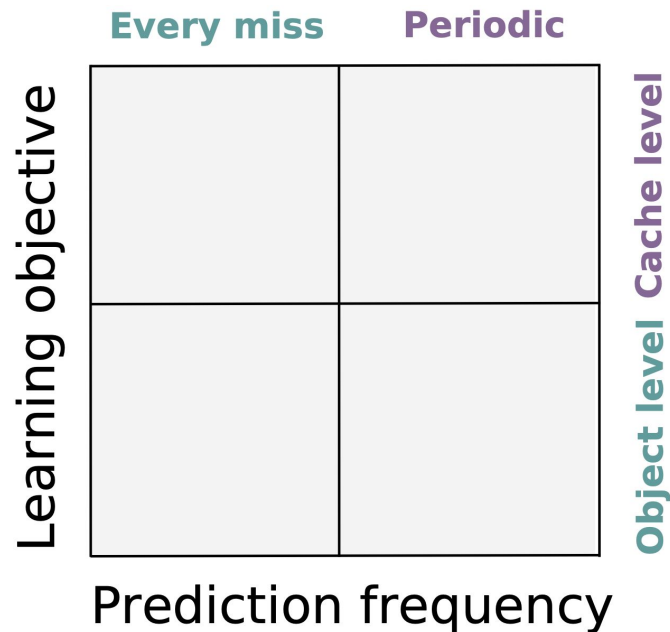
Smart Caches

LeCaR · LRB · LHD · 3L-Cache ·
GL-Cache

Smart: adapts to workload, often
beating static heuristics.

If smart caches are so good, why does production
still run plain heuristics?

Taxonomy of Smart Caches



Let's try to classify prior smart caches along these four quadrants!

First Axis: Learning Granularity

Smart caches adapt to workload in different ways:

Object Level: Scores objects in the cache (LRB)



Score: 10



Score: 50

Cache Level: Adjusts cache level parameters (ARC)



First Axis: Learning Granularity

Object Level: Scoring individual objects in the cache

Objective Mismatch

Object	A	B
Future Reuse Distance (RD)	1000	50

Ideally evict A

Unpredictability

Model A (lower loss)

Object	A	B
Predicted RD	40	50
L2 Loss	921,600	
Eviction Target	B	

Model B (fewer misses)

Object	A	B
Predicted RD	4000	60
L2 Loss	9,000,100	
Eviction Target	A	

Model Objective: Minimize reuse distance L2 loss

Cache Objective: Minimize miss ratio

First Axis: Learning Granularity

Cache Level: Optimizes cache level parameters



LRU (New)

LRU (Frequent)

Direct Objective

Directly optimizes for hit ratio

Magic Parameters

First Axis: Learning Granularity

Cache Level: Optimizes cache level parameters



LRU (New)

LRU (Frequent)

Direct Objective

Directly optimizes for hit ratio

Magic Parameters

Contains tunable parameters, e.g. how often is an update needed?

Second Axis: Prediction Frequency

Smart caches have various prediction frequency:

Every Miss

Makes a prediction at every miss.

Most ML based algorithms do this!

Instability

Overhead

Periodically

Reuses prediction for sometime.

Smart caches rarely do this.

**Low
Overhead**

**Stale
Prediction**

Taxonomy of Smart Caches

		Every miss	Periodic
Learning objective	Object level		
		Opaque and high overhead	
Prediction frequency			

1. Reacts on every miss and gathers information about every object in the cache.

Taxonomy of Smart Caches

		Every miss	Periodic
Learning objective	Object level		
		LRB LHD 3L-Cache	Still Opaque
		Prediction frequency	

1. Reacts on every miss and gathers information about every object in the cache.
2. Reduces overhead but still tracks every object in the cache.

Taxonomy of Smart Caches

		Every miss	Periodic
Learning objective	Object level	Less Opaque, Noisy	
		LRB LHD 3L-Cache	GL-Cache
		Prediction frequency	

1. Reacts on every miss and gathers information about every object in the cache.
2. Reduces overhead but still tracks every object in the cache.
3. Manipulates feature knobs, easier to reason about. Still chases noise.

Taxonomy of Smart Caches

Learning objective	Every miss	Periodic
	Object level	Cache level
	ARC LeCaR	Less Opaque, Robust
	LRB LHD 3L-Cache	GL-Cache
Prediction frequency		

1. Reacts on every miss and gathers information about every object in the cache.

2. Reduces overhead but still tracks every object in the cache.

3. Manipulates feature knobs, easier to reason about. Still chases noise.

There's an empty quadrant!

Introducing Learning Augmented Heuristics MaDSys

		Every miss	Periodic
Learning objective	Object level	ARC LeCaR	Learning-Augmented Heuristics (S4-FIFO)
		LRB LHD 3L-Cache	GL-Cache
Prediction frequency			

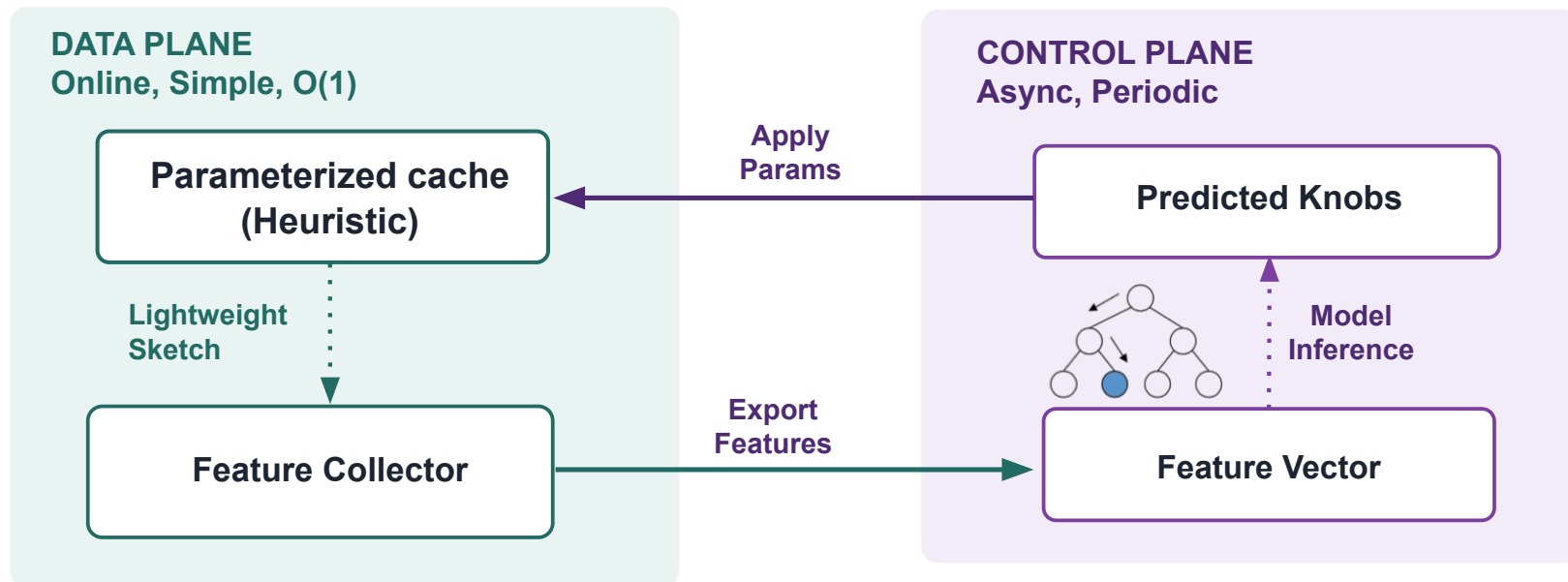
LAH bridges this gap:

- Work on cache level features
- Performs periodic prediction

LAH is as efficient as a heuristic, yet smart, and non-opaque with minimal overhead

LAH: Control-Data Plane Separation

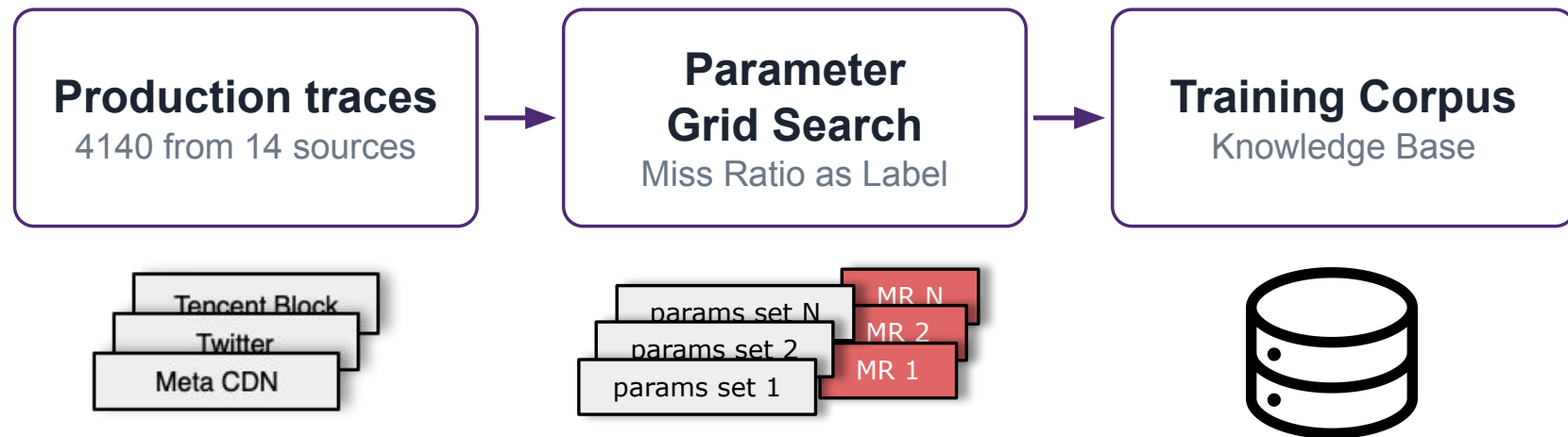
Use a pre-trained model to choose the heuristic's parameters.



Achieves **adaptivity** while maintaining **simplicity** and **performance**!

LAH: Model Pretraining

Creating a foundational model for caching



Cache-level features are workload oblivious, which allows us to accumulate knowledge across thousands of traces.

S4-FIFO: LAH in action

We apply principles of LAH on S3-FIFO, a SOTA eviction heuristic.

It utilizes 3 FIFO queues to effectively filter out 1-hit-wonders.

We obtain a total of 5 tunable parameters from the heuristic.

Table 2: Hyperparameters of the S4-FIFO.

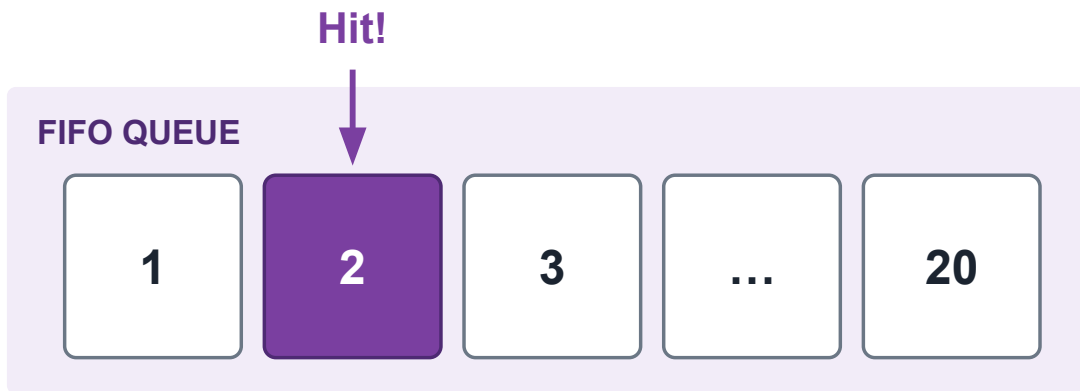
Symbol	Physical Meaning	Default	Learning range
ρ_S	Small queue size ratio	0.10	$[0.05, 0.90]$

Model will learn to output the best suited parameters for each workload.

S4-FIFO: Cache Level Features

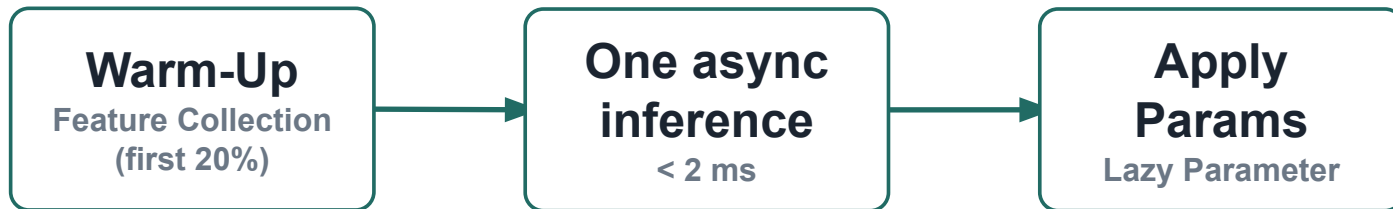
We collect 73 features from S4-FIFO, most of which are hit histograms.

Features	Dimension
Small histogram	20
Main histogram	20
Ghost histogram	20
Small hits ratio	1
Main hits ratio	1
Ghost hits ratio	1
Log cache size	1
Utility gap	1
Filtering efficiency	1
Ghost pressure	1
Tail heaviness	1
Decay rate	1
One-hit ratio	1
Unique ratio	1
Scan intensity	1
Thrashing risk	1



Features come from cheap atomic counters

S4-FIFO: Evaluation



5,175

production traces · 14 sources · block,
KV, CDN

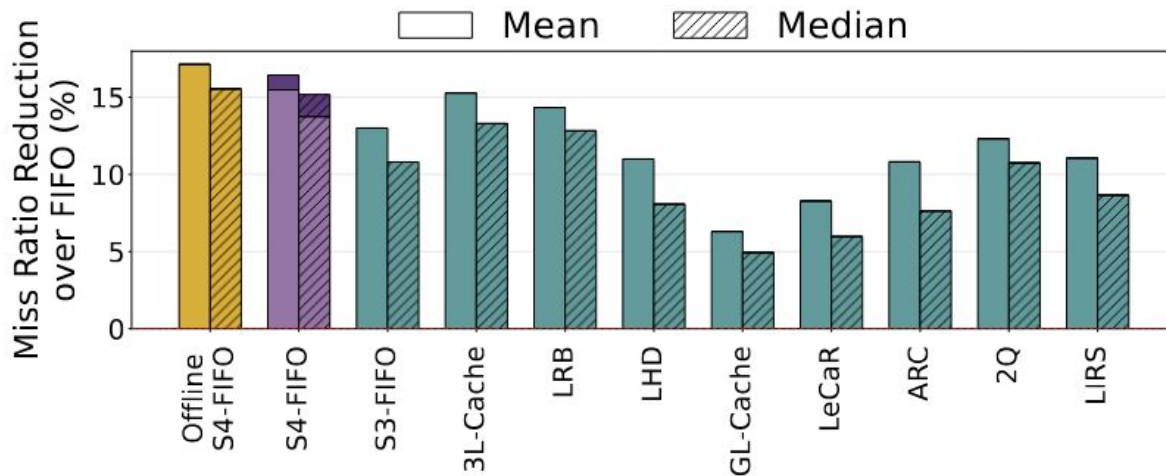
4,140 train / 1,035 test

one pre-trained model, evaluated
zero-shot

Four questions:

- **Efficiency:** Miss ratio reduction (MRR) over FIFO
- **Robustness:** Worst case performance
- **Throughput:** Reqs/s in CacheLib
- **Interpretability:** LLM interpretability

Efficiency: Beats other smart caches



+26%

MRR over S3-FIFO

+8%

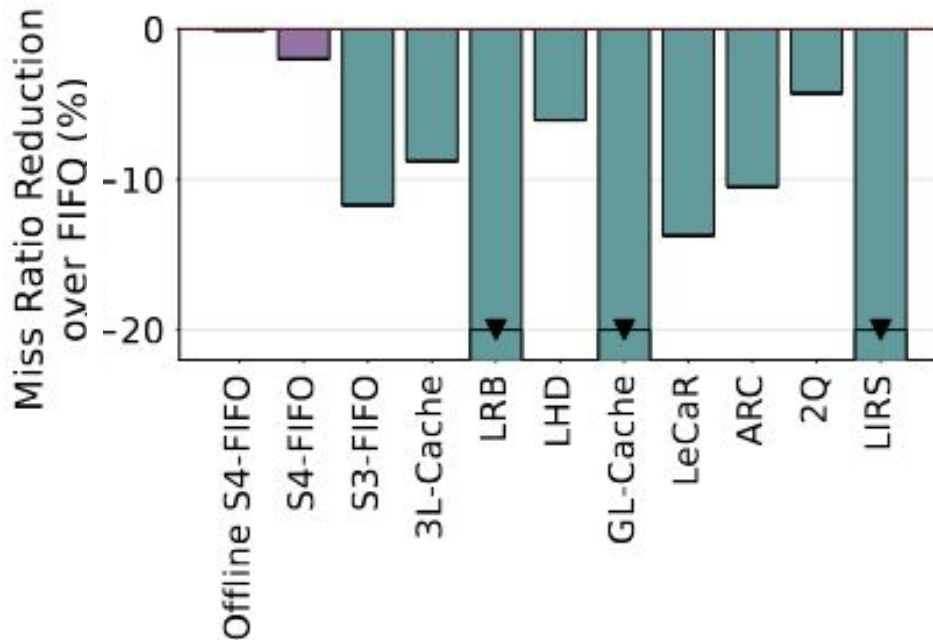
MRR over 3L-Cache

≈ Oracle

Predicted knobs near optimal

Robustness: Stable performance

Worst case performance over the test set

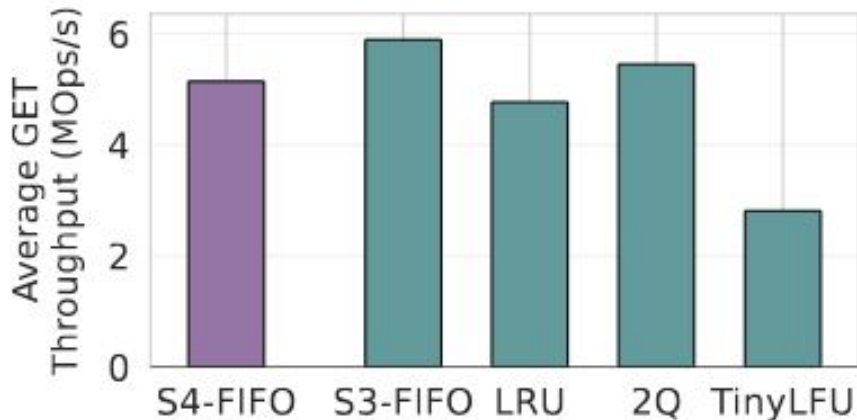


Efficient, yet robust

Every prior smart cache does worse than FIFO on its worst trace (up to 72%).

Throughput: Heuristic level speed

Evaluated on Cachelib with other eviction heuristics



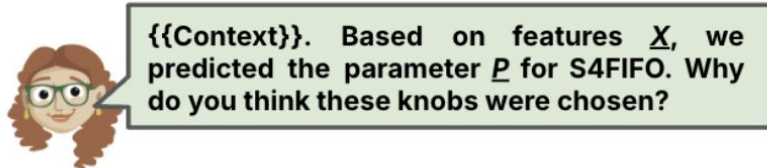
Critical Path Stays $O(1)$

Feature collections are lightweight counter increments.

Model overhead stays off the critical path.

Interpretability: Not an opaque smart caches

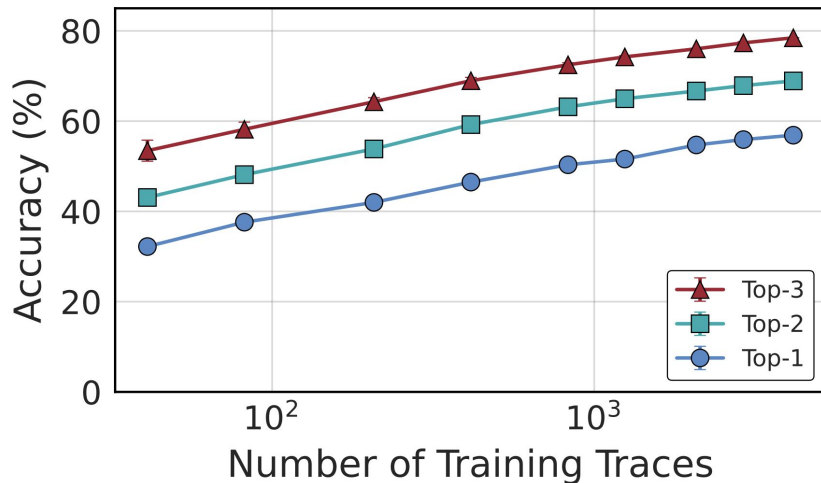
An LLM can reason through the predicted choice



Smart, yet Interpretable

Cache level features and tunable parameters carry semantically meaningful information.

Pretrained Model: Scaling



Welcome to
contribute traces!

Our Vision: A pretrained general
caching model that generalizes to
different workload

Learning and heuristics do not need to be competing approaches.

Smart

allows heuristics to
be workload
adaptive.

Simple & Fast

every cache operation
remains $O(1)$

Robust

Better worst case
performance

Interpretable

Features and knobs
semantically
meaningful

<https://libcachesim.com>

<https://github.com/cacheMon/osdi26-s4-fifo>

Object Level: Scoring individual objects in the cache

Objective Mismatch

A

Value: 1000

B

Value: 50

Unpredictability

C

Value: 70

D

Value: 500

Predicted scores are opaque, making it a risk to
deploy at scale.

Object Level: Scoring individual objects in the cache



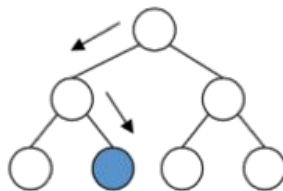
LRB: Scores object on **reuse distance**. Objects with low utility will be evicted.



Value: 1000
(Evicted)



Value: 50



A model is trained to minimize L2 Loss.

Smart Caches: Learning Granularity

Smart caches adapts to workload in different ways:

Object Level: Scores objects in the cache



Value: 10



Value: 50

Cache Level: Adjusts cache level parameters



Smart Caches: Learning Granularity

Object Level: Scoring individual objects in the cache

Objective Mismatch

Object	A	B
Future Reuse Distance (RD)	1000	50

Ideally evict A

Unpredictability

Model A (lower loss)

Object	A	B
Predicted RD	40	50
L2 Loss	921,600	
Eviction Target	B	

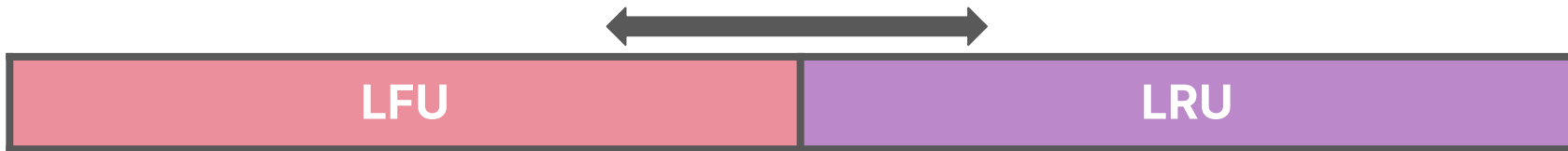
Model B (fewer misses)

Object	A	B
Predicted RD	4000	60
L2 Loss	9,000,100	
Eviction Target	A	

Optimizing object loss does not naturally directly translate to miss ratio reduction.

Smart Caches: Learning Granularity

Cache Level: Optimizes cache level parameters



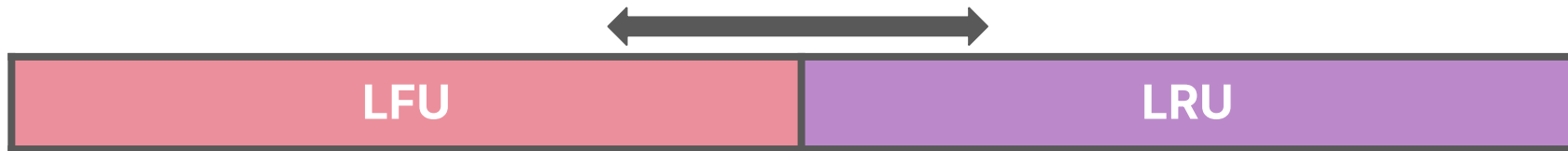
Direct Objective

Directly optimizes for hit ratio

Magic Parameters

Smart Caches: Learning Granularity

Cache Level: Optimizes cache level parameters



Direct Objective

Directly optimizes for hit ratio

Magic Parameters

Contains tunable parameters, e.g. how often is an update needed?

Smart Caches: Prediction Frequency

Smart caches have various prediction frequency:

Every Miss

Makes a prediction at every miss.

Most ML based algorithms do this!

Instability

Overhead

Periodically

Reuses prediction for sometime.

Not much work here.

**Low
Overhead**

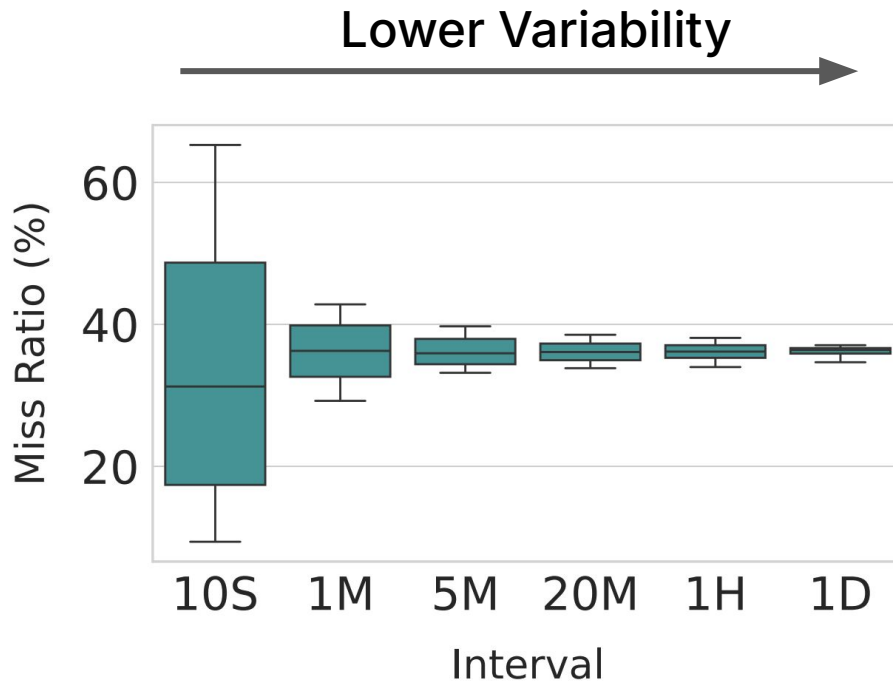
**Stale
Prediction**

Smart Caches: Prediction Frequency

Reaction: On every miss

Instability

Miss ratio exhibits major variation at finer granularities, causing noisy adaptation



Second Axis: Prediction Frequency

Reaction: On every miss

Instability

Overhead

Miss ratio exhibits major variation at finer granularities, causing noisy adaptation

